

WEB APPLICATION & API SECURITY ASSESSMENT

Security Assessment of OWASP Juice Shop

A Practical Penetration Testing and Vulnerability Assessment

Prepared By:

Sesadi Bandara

Degree Programme:

BSc in Information and Communication Technology

Specialization: Network and Security Technologies

University:

South Eastern University of Sri Lanka

Assessment Type:

Web Application & API Security Assessment

Target Application:

OWASP Juice Shop

Testing Environment:

Local Authorized Laboratory Environment

Tools Used:

Podman | Burp Suite | Nmap | Web Browser

Date:

September 2026

Contents

Content of Tables.....	4
Content of Figures.....	5
1. Executive Summary	6
2.Introduction.....	6
2.1 Background	6
2.2 Assessment Purpose	7
2.3 Objectives.....	7
2.4 Scope.....	8
3. Assessment Environment.....	9
3.1 Target Application.....	9
3.2 Laboratory Architecture	9
3.3 Tools Used	10
4. Methodology	10
4.1 Reconnaissance.....	11
4.2 HTTP Traffic Interception	11
4.3 API Enumeration	12
4.4 Endpoint Classification	13
4.5 Vulnerability Assessment.....	14
4.6 Validation	14
4.7 Risk Assessment	14
5. OWASP Security Mapping.....	15
6. Reconnaissance and Enumeration Results	16
6.1 Target Verification	16
6.2 Port and Service Discovery	17
6.3 Web Application Enumeration	17
6.4 API Endpoint Enumeration	19

7. Vulnerability Assessment Findings.....	21
7.1 BOLA / IDOR.....	21
7.2 Broken Authentication.....	24
7.3 Broken Function-Level Authorization (BFLA).....	27
7.4 Injection	31
7.5 Security Misconfiguration.....	34
8. Risk Assessment and CVSS Scoring.....	36
8.1 Risk Classification.....	36
8.2 Risk Registration.....	36
9. Remediation Recommendations	38
10. Conclusion	39

Content of Tables

Table 1 - Target Information	9
Table 2 - Tools Used	10
Table 3 - Burp suite HTTP History Records	13
Table 4 - OWASP security categories	15
Table 5 - BOLA Test Records	22
Table 6 - Broken Authentication Test Records	25
Table 7 - BFLA Test Records	28
Table 8 - Injection Test Records	31
Table 9 - Risk Classification	36
Table 10 - Risk Registration	36
Table 11 - CVSS scores	38

Content of Figures

Figure 1 — Laboratory Assessment Architecture	10
Figure 2 - Burp Proxy listener configuration	12
Figure 3 - OWASP Juice Shop Application	16
Figure 4 - Nmap Service Discovery.....	17
Figure 5 - Web Application Enumeration	18
Figure 6 - Burp HTTP History	18
Figure 7 - Burp Proxy listener configuration	19
Figure 8 - Burp Target Site Map	19
Figure 9 - Example API request.....	20
Figure 10 - Example API response	20
Figure 11 - — BOLA Baseline Request.....	22
Figure 12 - BOLA Modified Request	23
Figure 13 - BOLA Response Comparison	24
Figure 14 - Authentication Request Captured Using Burp Suite.....	25
Figure 15 - Authentication Response and Session Information.....	26
Figure 16 - Invalid Authentication Attempt and Server Response	26
Figure 17 - Authenticated Baseline.....	27
Figure 18 - Unauthenticated Test.....	27
Figure 19 - Administrative user accessing a privileged application function	28
Figure 20 - API request generated while accessing the privileged function.....	29
Figure 21 - Privileged API function successfully invoked by an administrator	29
Figure 22 - Replay of the privileged API request using a standard-user session.....	30
Figure 23 - Standard user successfully invoking a function restricted to administrative users	30
Figure 24 - Normal product search request and response captured using Burp Suite	32
Figure 25 - Injection test payload submitted through the API parameter	33
Figure 26 - API response following injection test	34
Figure 27 - HTTP response headers observed during security configuration assessment.....	35
Figure 28 - Detailed server error response exposing internal application information	35
Figure 29 - Security-related HTTP header configuration identified during testing.....	36

1. Executive Summary

This report presents the results of a Web Application and API Security Assessment conducted against the OWASP Juice Shop application within an authorized local laboratory environment.

The primary objective of the assessment was to identify, analyze, validate, and document security weaknesses affecting the application's web interface and API layer. The assessment follows a structured penetration-testing methodology consisting of reconnaissance, enumeration, HTTP traffic analysis, API endpoint discovery, vulnerability assessment, validation, risk evaluation, and remediation planning.

The assessment environment was deployed using Podman, while Burp Suite was used to intercept and analyze HTTP traffic generated by the application. Nmap was used during the reconnaissance phase to identify exposed services and ports.

Initial assessment activities successfully established the target environment, verified application accessibility, identified the exposed application service, and began enumerating API endpoints through Burp Suite HTTP History and Site Map functionality.

Particular attention was given to vulnerabilities associated with the OWASP API Security Top 10 and common web application security risks, including Broken Object Level Authorization (BOLA/IDOR), broken authentication, broken function-level authorization, excessive data exposure, injection, and security misconfiguration.

The assessment identified and validated security weaknesses affecting authorization, authentication, injection handling, and security configuration. Confirmed findings are documented with supporting evidence, risk ratings, CVSS scores, and remediation recommendations.

2. Introduction

2.1 Background

Web applications have become an essential component of modern information systems. Organizations increasingly depend on web-based applications and application programming interfaces (APIs) to provide services, process information, communicate with databases, and integrate different software systems.

Although APIs and web applications provide significant functionality and flexibility, vulnerabilities in authentication, authorization, input validation, session management, and information disclosure can expose applications to security threats.

APIs are particularly important from a security perspective because they frequently provide direct access to application functionality and backend data. A poorly protected API may

allow unauthorized users to access, modify, or delete resources that they should not be permitted to access.

The Open Worldwide Application Security Project (OWASP) provides widely recognized security guidance for web applications and APIs. OWASP identifies common security weaknesses and provides testing methodologies that can be used by security professionals during vulnerability assessments.

OWASP Juice Shop was selected as the target application for this project because it is intentionally designed to contain common web application security vulnerabilities. It provides a safe and controlled environment for learning penetration-testing techniques and demonstrating security assessment methodologies.

This project applies practical web application and API security testing techniques to the Juice Shop environment while following an evidence-based assessment methodology.

2.2 Assessment Purpose

The primary purpose of this assessment is to evaluate the security of the OWASP Juice Shop web application and its API components within a controlled laboratory environment.

The assessment is also intended to demonstrate practical knowledge of:

- Web application reconnaissance
- API enumeration
- HTTP traffic analysis
- Authentication testing
- Authorization testing
- Vulnerability identification
- Manual vulnerability validation
- Risk assessment
- CVSS-based severity classification
- Security remediation
- Professional penetration-testing reporting

The project provides practical experience in applying theoretical cybersecurity concepts to a real application security testing environment.

2.3 Objectives

The main objectives of this assessment are:

- 1.To deploy and verify an intentionally vulnerable web application within a controlled environment.
- 2.To identify exposed services and application entry points.
- 3.To intercept and analyze HTTP communication between the client and server.
- 4.To discover and enumerate API endpoints.
- 5.To identify API parameters, HTTP methods, authentication mechanisms, and responses.
- 6.To classify endpoints according to potential security risks.
- 7.To assess the application for BOLA/IDOR vulnerabilities.
- 8.To assess authentication and session-management mechanisms.
- 9.To assess function-level authorization controls.
10. To identify excessive exposure of sensitive information.
11. To identify potential injection vulnerabilities.
12. To assess relevant security misconfigurations.
13. To validate identified vulnerabilities using reproducible evidence.
14. To assign appropriate severity and CVSS scores to confirmed findings.
15. To provide practical remediation recommendations.

2.4 Scope

In Scope

The following components are included in the assessment:

- OWASP Juice Shop web application
- Web application functionality
- REST/API endpoints
- HTTP requests and responses
- Authentication functionality
- Authorization mechanisms
- User-specific resources
- Object identifiers
- API parameters
- JSON request and response bodies
- Session information
- Security-related HTTP headers
- Application error responses

Out of Scope

The following are outside the scope of this assessment:

- Real-world external systems
- Third-party websites

- Other users' systems
 - Production applications
 - Denial-of-service attacks against external infrastructure
 - Social engineering
 - Physical security
 - Attacks against systems not belonging to the laboratory environment
- All testing is restricted to the authorized OWASP Juice Shop laboratory target.

3. Assessment Environment

3.1 Target Application

The target application selected for this assessment is OWASP Juice Shop.

Juice Shop is an intentionally vulnerable web application designed for security training and penetration-testing practice. It includes a variety of common web application and API security weaknesses.

The application was deployed locally using Podman.

Target Information

Table 1 - Target Information

Parameter	Details
Target Application	OWASP Juice Shop
Target URL	http://localhost:3000
Protocol	HTTP
Port	3000
Deployment Platform	Podman
Assessment Type	Web Application & API Security Assessment
Environment	Local authorized laboratory

3.2 Laboratory Architecture

The assessment was conducted within a controlled local environment.

The general architecture is:

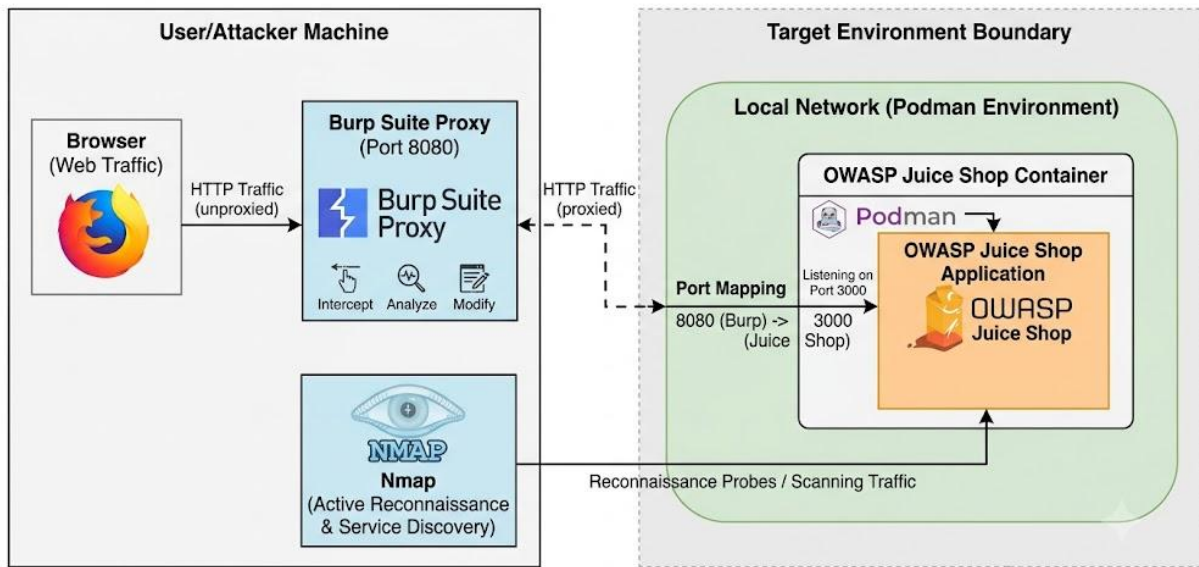


Figure 1 — Laboratory Assessment Architecture

The browser is used to interact with the Juice Shop application. Burp Suite acts as an HTTP interception and analysis point, allowing requests and responses to be inspected. The Juice Shop application is hosted within a Podman container and exposed through TCP port 3000.

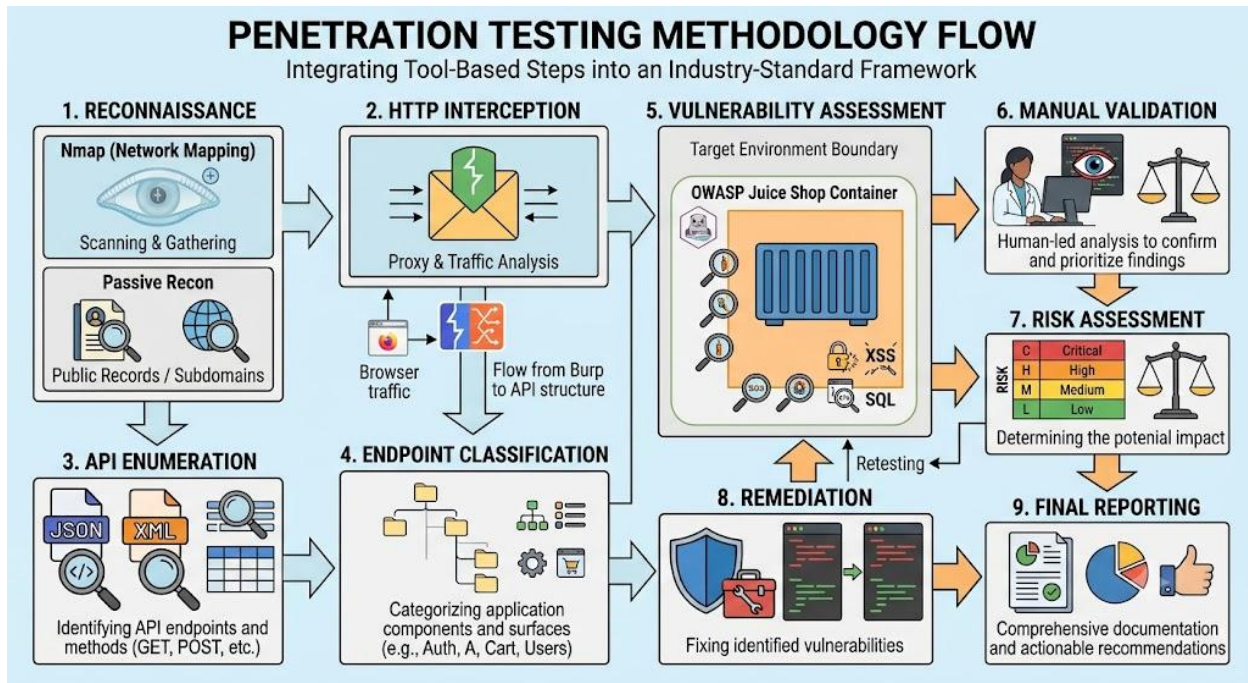
3.3 Tools Used

Table 2 - Tools Used

Tool	Purpose
Podman	Container deployment and management
OWASP Juice Shop	Intentionally vulnerable target
Burp Suite Community Edition	Web application security testing
Burp Proxy	HTTP traffic interception
Burp HTTP History	Request and response analysis
Burp Target/Site Map	Application and API enumeration
Burp Repeater	Manual request modification and validation
Nmap	Port and service discovery
Web Browser	Application interaction

4. Methodology

The assessment follows a structured penetration-testing methodology.



4.1 Reconnaissance

The reconnaissance phase establishes information about the target before detailed vulnerability testing begins.

The following activities were performed:

- Target identification
- Application accessibility verification
- Port identification
- Service identification
- Initial application observation

Nmap was used to examine the target service on TCP port 3000.

The command used was:

```
nmap -sV -p 3000 localhost
```

The results are documented in Section 6.2.

4.2 HTTP Traffic Interception

Burp Suite was used to intercept HTTP traffic generated by the web browser.

The Burp Proxy listener was configured on the local interface:

127.0.0.1:8080

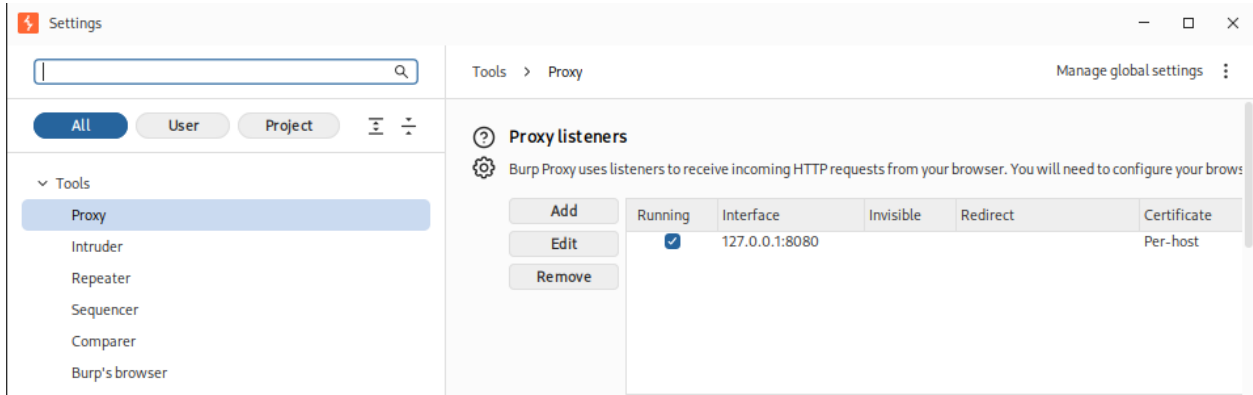


Figure 2 - Burp Proxy listener configuration

The browser was then used to access:

http://localhost:3000

The captured requests were analyzed through:

Proxy → HTTP history

The following information was reviewed:

- HTTP method
- URL/path
- Query parameters
- Request headers
- Cookies
- Authorization headers
- Request body
- Response status code
- Response headers
- Response body

4.3 API Enumeration

API enumeration was performed by interacting normally with the application while Burp Suite recorded the resulting HTTP traffic.

The following application actions were used to generate traffic:

- Opening application pages
- Searching for products

- Viewing products
- Authentication-related activities
- Basket-related activities
- Other normal application interactions

Captured requests were examined through Burp HTTP History and Target Site Map.

The following information was recorded:

Table 3 - Burp suite HTTP History Records

Information	Purpose
HTTP Method	Determine request operation
Endpoint	Identify application/API resource
Parameters	Identify user-controlled input
Authentication	Determine access requirements
Status Code	Analyze server behavior
Response	Identify returned information

4.4 Endpoint Classification

Discovered endpoints were categorized according to their potential security impact.

Examples of high-priority endpoint categories include:

- User/account resources
- Basket resources
- Order resources
- Authentication endpoints
- Administrative functionality
- Object-based endpoints
- Search/query endpoints
- API endpoints returning sensitive information

The classification process helps prioritize manual security testing.

4.5 Vulnerability Assessment

The vulnerability assessment focuses on security weaknesses relevant to the discovered attack surface.

The primary testing categories are:

- BOLA/IDOR
- Broken Authentication
- Broken Function-Level Authorization
- Excessive Data Exposure
- Injection
- Security Misconfiguration

Each potential vulnerability is assessed using a controlled and repeatable methodology.

4.6 Validation

A suspected vulnerability is not immediately considered a confirmed finding.

Validation requires:

1. Establishing normal application behavior.
2. Recording a baseline request.
3. Modifying the relevant security-related parameter.
4. Repeating the request.
5. Comparing responses.
6. Determining whether unauthorized behavior occurred.
7. Capturing evidence.

For example, during BOLA testing, only the object identifier should be modified while the authentication/session context remains unchanged.

This ensures that the test isolates object-level authorization.

4.7 Risk Assessment

Confirmed vulnerabilities are evaluated according to:

- Likelihood
- Technical impact
- Business impact
- Exploitability
- Required privileges
- User interaction
- Confidentiality impact
- Integrity impact
- Availability impact

CVSS will be used to provide a standardized technical severity score where appropriate.

5. OWASP Security Mapping

The assessment activities are mapped to relevant OWASP security categories.

Table 4 - OWASP security categories

Assessment Area	OWASP Mapping
BOLA / IDOR	OWASP API Security Top 10 – Broken Object Level Authorization
Authentication	OWASP API Security Top 10 – Broken Authentication
Object/property authorization	OWASP API Security Top 10 – Broken Object Property Level Authorization
Resource consumption	OWASP API Security Top 10 – Unrestricted Resource Consumption
Function-level authorization	OWASP API Security Top 10 – Broken Function Level Authorization
SSRF	OWASP API Security Top 10 – Server-Side Request Forgery
Security configuration	OWASP API Security Top 10 – Security Misconfiguration
API inventory	OWASP API Security Top 10 – Improper Inventory Management
Unsafe API consumption	OWASP API Security Top 10 – Unsafe Consumption of APIs

Injection	OWASP Web Application Security Testing / Injection-related risks
-----------	--

The confirmed findings are mapped to the relevant OWASP security categories based on the results of technical validation

6. Reconnaissance and Enumeration Results

6.1 Target Verification

The Juice Shop application was successfully deployed using Podman and accessed through the local browser.

Target:

<http://localhost:3000>

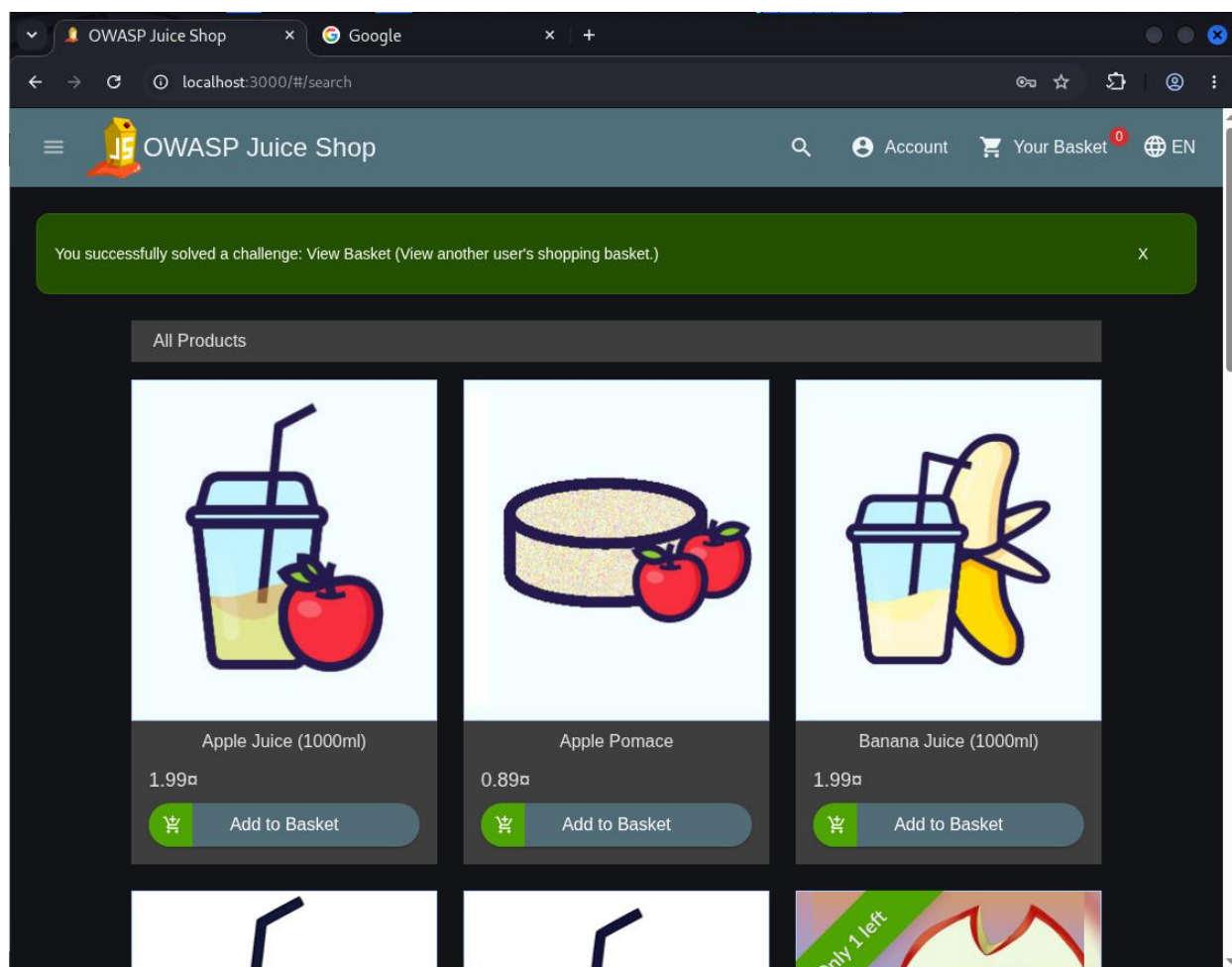


Figure 3 - OWASP Juice Shop Application

Figure 3 demonstrates that the OWASP Juice Shop application was successfully deployed and accessible within the local laboratory environment.

6.2 Port and Service Discovery

Nmap was used to identify the service exposed by the Juice Shop application.

Command:

```
nmap -sV -p 3000 localhost
```

Results

```
(user@kali)-[~]
$ nmap -sV -p 3000 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-01 11:44 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000048s latency).
Other addresses for localhost (not scanned): ::1
PORT      STATE SERVICE VERSION
3000/tcp  open  ppp?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port3000-TCP:V=7.99%I=7%D=9/1%Time=6A966D61%P=x86_64-pc-linux-gnu%r(Get
SF:Request,2668,"HTTP/1.1\x20200\x200K\r\nAccess-Control-Allow-Origin:\x2
SF:0*\r\nX-Content-Type-Options:\x20nosniff\r\nX-Frame-Options:\x20SAMEOR
```

Figure 4 - Nmap Service Discovery

The result establishes the network service through which the Juice Shop application is accessible.

6.3 Web Application Enumeration

Initial web application enumeration was performed through normal interaction with Juice Shop.

The following application areas were examined:

- Home page
- Product listings
- Product search
- Product details
- Login functionality
- Basket functionality
- Other accessible application functionality

Burp Suite was used to record the HTTP requests generated during these activities.

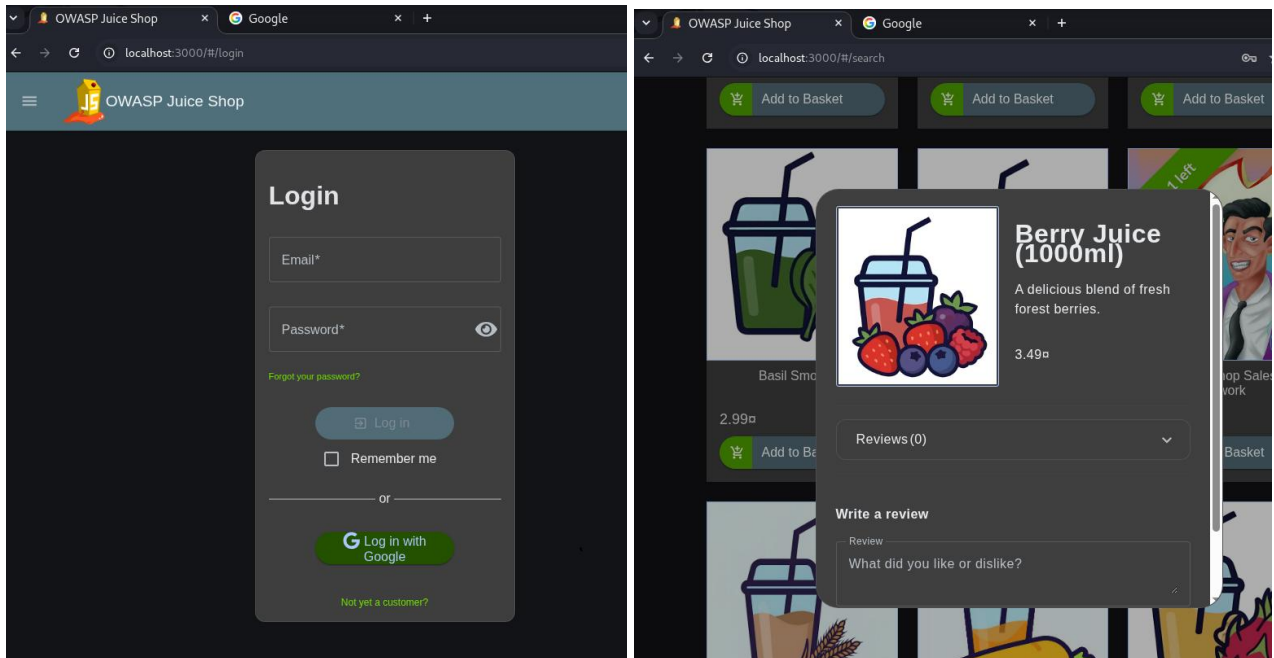


Figure 5 - Web Application Enumeration

Burp Suite Community Edition v2026.7.3 - Temporary Project													
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover													
Intercept HTTP History WebSockets history Match and replace Proxy settings													
Filter settings: Hiding CSS and image content; hiding specific extensions													
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
1	http://localhost:3000	GET	/			200	9860	HTML		OWASP Juice Shop		127.0.0.1	
10	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	326	text	io/			127.0.0.1	
11	http://localhost:3000	GET	/assets/18/en.json			200	35940	JSON	json			127.0.0.1	
12	http://localhost:3000	GET	/rest/admin/application-version			200	404	JSON				127.0.0.1	
13	http://localhost:3000	GET	/rest/admin/application-configuration			200	23966	JSON				127.0.0.1	
14	http://localhost:3000	GET	/rest/languages			200	4603	JSON				127.0.0.1	
15	http://localhost:3000	GET	/api/challenges/?name=Score%20Board			200	862	JSON				127.0.0.1	
16	http://localhost:3000	GET	/api/quantity/			200	8016	JSON				127.0.0.1	
18	http://localhost:3000	GET	/rest/admin/application-version			304	304					127.0.0.1	
20	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	262	text	io/			127.0.0.1	
21	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	
22	http://localhost:3000	POST	/rest/products/search?q=			200	16967	JSON				127.0.0.1	
23	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	215	HTML	io/			127.0.0.1	
24	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	
25	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	
26	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	
27	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	230	text	io/			127.0.0.1	
28	http://localhost:3000	GET	/api/challenges/?name=Score%20Board			304	305					127.0.0.1	
29	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	
46	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websocket&sid=k...			101	129		io/			127.0.0.1	
47	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	326	text	io/			127.0.0.1	
48	http://localhost:3000	GET	/			304	392					127.0.0.1	
49	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=Q1RdM...			200	326	text	io/			127.0.0.1	
Request Response													
Pretty Raw Hex													
1	GET /rest/basket/6 HTTP/1.1												
2	Host: localhost:3000												
3	sec-ch-ua-platform: "Linux"												
4	Authorization: Bearer												
5	Accept-Language: en-US,en;q=0.9												
6	Accept: application/json, text/plain, */*												
7	sec-ch-ua: "Not.A.Brand";v="8", "Chrome";v="150"												
8	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36												
9	sec-ch-ua-mobile: ?0												
10	Sec-Petch-Site: same-origin												
11	Sec-Petch-Mode: cors												
12	Sec-Petch-Dest: empty												

Figure 6 - Burp HTTP History

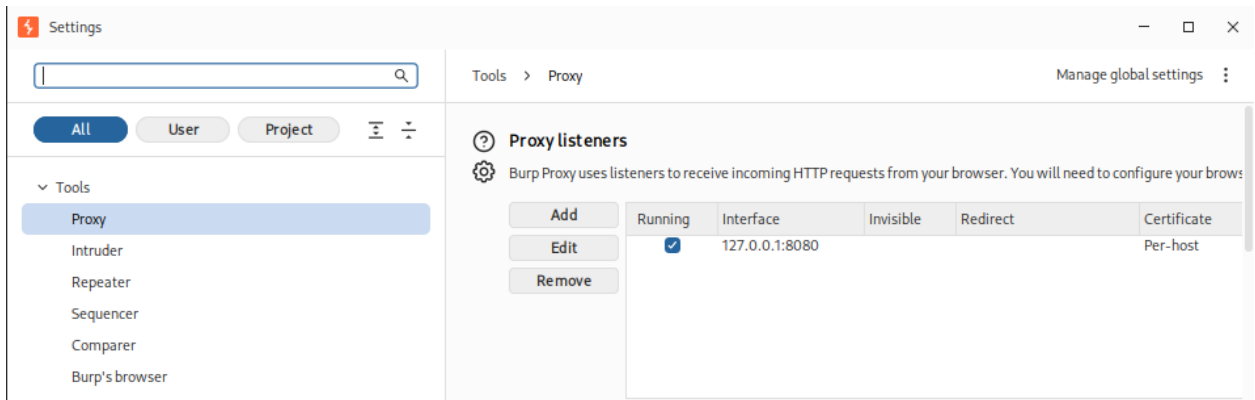


Figure 7 - Burp Proxy listener configuration

The HTTP History provides an overview of requests generated by the application.

6.4 API Endpoint Enumeration

API endpoints were identified by reviewing the captured HTTP traffic.

The API endpoints observed during testing were reviewed and documented based on the HTTP traffic captured using Burp Suite.

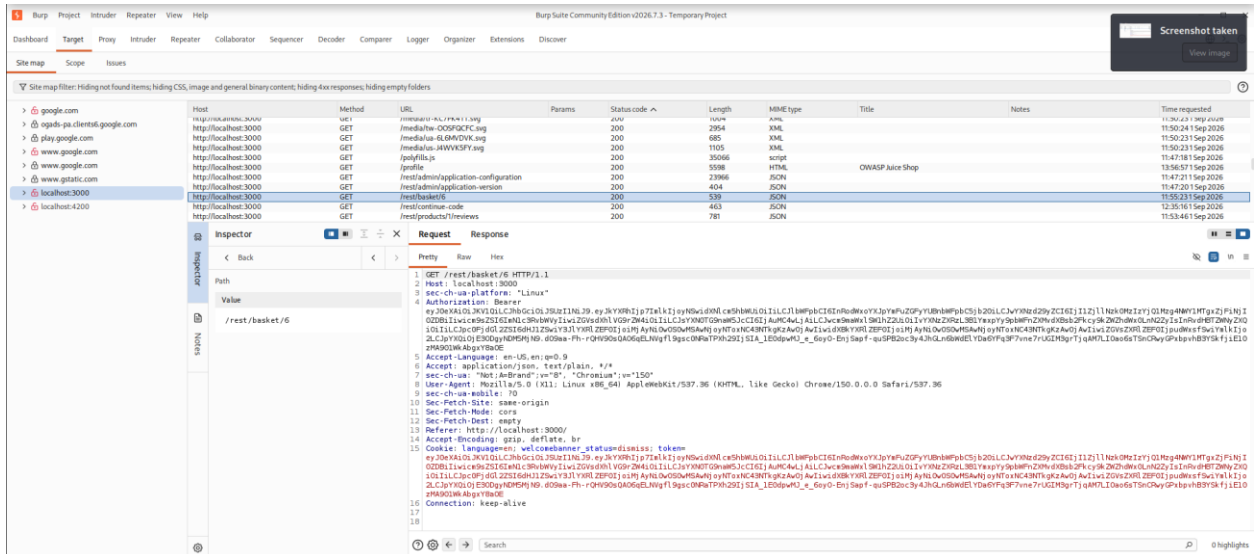


Figure 8 - Burp Target Site Map

Inspector	Request		Response
	Pretty	Raw	Hex
Notes	<pre> 1 POST /api/Users/ HTTP/1.1 2 Host: localhost:3000 3 Content-Length: 271 4 sec-ch-ua-platform: "Linux" 5 Accept-Language: en-US,en;q=0.9 6 Accept: application/json, text/plain, */* 7 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150" 8 Content-Type: application/json 9 sec-ch-ua-mobile: ?0 10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36 11 Origin: http://localhost:3000 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: http://localhost:3000/ 16 Accept-Encoding: gzip, deflate, br 17 Cookie: language=en; welcomebanner_status=dismiss 18 Connection: keep-alive 19 20 { "email": "thulharibandara@gmail.com", "password": "123456789", "passwordRepeat": "123456789", "securityQuestion": { "id": 1, "question": "Your eldest siblings middle name?", "createdAt": "2026-09-01T06:11:28.864Z", "updatedAt": "2026-09-01T06:11:28.864Z" } } </pre>		

Figure 9 - Example API request

Inspector

Notes

Request

Response

Pretty

Raw

Hex

Render

1 HTTP/1.1 201 Created

2 Access-Control-Allow-Origin: *

3 X-Content-Type-Options: nosniff

4 X-Frame-Options: SAMEORIGIN

5 Feature-Policy: payment 'self'

6 X-Recruiting: /#/jobs

7 Location: /api/Users/25

8 Content-Type: application/json; charset=utf-8

9 Content-Length: 316

10 ETag: W/"13c-88XfjwdouYnSOWoVo6Nse0x3FD0"

11 Vary: Accept-Encoding

12 Date: Tue, 01 Sep 2026 06:25:14 GMT

13 Connection: keep-alive

14 Keep-Alive: timeout=5

15

16 {

17 "status": "success",

18 "data": {

19 "username": "",

20 "role": "customer",

21 "deluxeToken": "",

22 "lastLoginIp": "0.0.0.0",

23 "profileImage": "/assets/public/images/uploads/default.svg",

24 "isActive": true,

25 "id": 25,

26 "email": "thulharibandara@gmail.com",

27 "updatedAt": "2026-09-01T06:25:14.759Z",

28 "createdAt": "2026-09-01T06:25:14.759Z",

29 "deletedAt": null

30 }

31 }

32 }

Figure 10 - Example API response

The Site Map provides a structured representation of resources discovered during the assessment.

7. Vulnerability Assessment Findings

This section contains only **validated security findings**.

A finding should not be marked as confirmed merely because the application is intentionally vulnerable.

Each finding will follow the structure:

- Finding ID
- Title
- Severity
- OWASP Mapping
- Affected Endpoint
- Description
- Testing Method
- Evidence
- Impact
- CVSS
- Remediation

7.1 BOLA / IDOR

Description

Broken Object Level Authorization (BOLA), commonly associated with IDOR-style vulnerabilities, occurs when an application fails to properly verify whether an authenticated user is authorized to access a specific object.

For example, an API request may contain an object identifier:

/resource/1

The assessment tests whether changing the identifier to another value allows unauthorized access.

Testing Method

1. Authenticate using a test account.
2. Identify an object-based API request.
3. Capture the request in Burp.
4. Send the request to Repeater.

Request

PrettyRawHex

1 GET /rest/basket/5 HTTP/1.1

2 Host: localhost:3000

3 sec-ch-ua-platform: "Linux"

4 Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpbImkIj0yNSwidXNlcm5hbWUiOiIiLCJlbWFBbCI6InRodWxoYXJpYmFuZG

5 Accept-Language: en-US,en;q=0.9

6 Accept: application/json, text/plain, */*

7 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"

8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36

9 sec-ch-ua-mobile: ?0

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: cors

12 Sec-Fetch-Dest: empty

13 Referer: http://localhost:3000/

14 Accept-Encoding: gzip, deflate, br

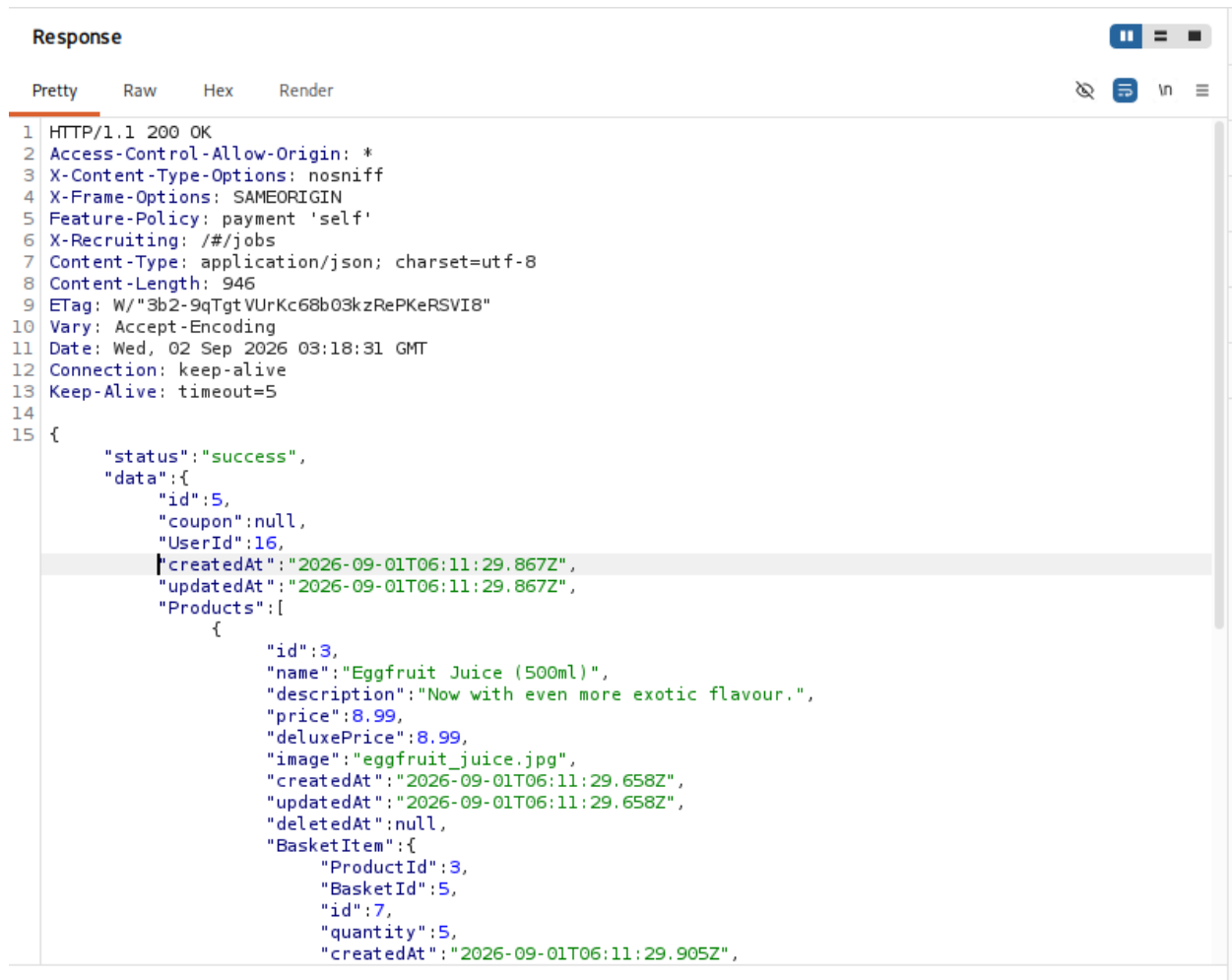
15 Cookie: language=en; welcomebanner_status=dismiss; token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpbImkIj0yNSwidXNlcm5hbWUiOiIiLCJlbWFBbCI6InRodWxoYXJpYmFuZG

16 Connection: keep-alive

17

Figure 12 - BOLA Modified Request

Request	Response
PrettyRawHexRender 1 HTTP/1.1 200 OK 2 Access-Control-Allow-Origin: * 3 X-Content-Type-Options: nosniff 4 X-Frame-Options: SAMEORIGIN 5 Feature-Policy: payment 'self' 6 X-Recruiting: /#/jobs 7 Content-Type: application/json; charset=utf-8 8 Content-Length: 154 9 ETag: W/"9a-5j7bmX/YeG/qEmy7TDSEoL4e0+M" 10 Vary: Accept-Encoding 11 Date: Tue, 01 Sep 2026 06:25:23 GMT 12 Connection: keep-alive 13 Keep-Alive: timeout=5 14 15 { 16 "status": "success", 17 "data": { 18 "id": 6, 19 "coupon": null, 20 "UserId": 25, 21 "createdAt": "2026-09-01T06:25:23.267Z", 22 "updatedAt": "2026-09-01T06:25:23.267Z", 23 "Products": [24] 25 } 26 }	



```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 946
9 ETag: W/"3b2-9qTgtVUrKc68b03kzRePKeRSVI8"
10 Vary: Accept-Encoding
11 Date: Wed, 02 Sep 2026 03:18:31 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": {
    "id": 5,
    "coupon": null,
    "userId": 16,
    "createdAt": "2026-09-01T06:11:29.867Z",
    "updatedAt": "2026-09-01T06:11:29.867Z",
    "Products": [
      {
        "id": 3,
        "name": "Eggfruit Juice (500ml)",
        "description": "Now with even more exotic flavour.",
        "price": 8.99,
        "deluxePrice": 8.99,
        "image": "eggfruit_juice.jpg",
        "createdAt": "2026-09-01T06:11:29.658Z",
        "updatedAt": "2026-09-01T06:11:29.658Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 3,
          "BasketId": 5,
          "id": 7,
          "quantity": 5,
          "createdAt": "2026-09-01T06:11:29.905Z",
```

Figure 13 - BOLA Response Comparison

Current Conclusion

No BOLA/IDOR vulnerability should be classified as confirmed until the test demonstrates unauthorized access to another user's object.

7.2 Broken Authentication

Assessment Objective

Determine whether authentication mechanisms can be bypassed, improperly implemented, or abused.

Tests

- Valid credentials
- Invalid credentials

- Invalid password
- Unknown account
- Missing authentication information
- Session/token behavior
- Unauthorized API access

Testing Records

Table 6 - Broken Authentication Test Records

Test	Authentication	Expected	Result
Baseline	Present	Protected resource accessible	HTTP/1.1 200 OK
Unauthenticated	Removed	Access denied	HTTP/1.1 401 Unauthorized

Evidence

132	http://localhost:3000	POST	/api/securityAnswers/	✓	201	651	JSON
133	http://localhost:3000	GET	/rest/admin/application-configuration		304	306	
134	http://localhost:3000	POST	/rest/user/login	✓	200	1198	JSON
135	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	304	303	
136	http://localhost:3000	GET	/rest/user/whoami		304	303	
137	http://localhost:3000	GET	/rest/basket/6		200	539	JSON
138	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	200	430	JSON
139	http://localhost:3000	GET	/rest/user/whoami		200	522	JSON
140	http://localhost:3000	GET	/rest/products/search?q=	✓	304	306	

Request	Response
Pretty Raw Hex <pre> 1 POST /rest/user/login HTTP/1.1 2 Host: localhost:3000 3 Content-Length: 60 4 sec-ch-ua-platform: "Linux" 5 Accept-Language: en-US,en;q=0.9 6 Accept: application/json, text/plain, */* 7 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150" 8 Content-Type: application/json 9 sec-ch-ua-mobile: ?0 10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36 11 Origin: http://localhost:3000 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: http://localhost:3000/ 16 Accept-Encoding: gzip, deflate, br 17 Cookie: language=en; welcomebanner_status=dismiss 18 Connection: keep-alive 19 20 { "email": "thulharibandara@gmail.com", "password": "123456789" }</pre>	

Figure 14 - Authentication Request Captured Using Burp Suite

against expected authorization controls.

Testing Table

Table 7 - BFLA Test Records

Test	Role	Method	Expected	Actual	Result
BFLA-01	Admin	GET	Allowed	200	Pass
BFLA-01	Normal User	GET	403	403	Pass
BFLA-02	Admin	POST	Allowed	200	Pass
BFLA-02	Normal User	POST	403	200	Fail

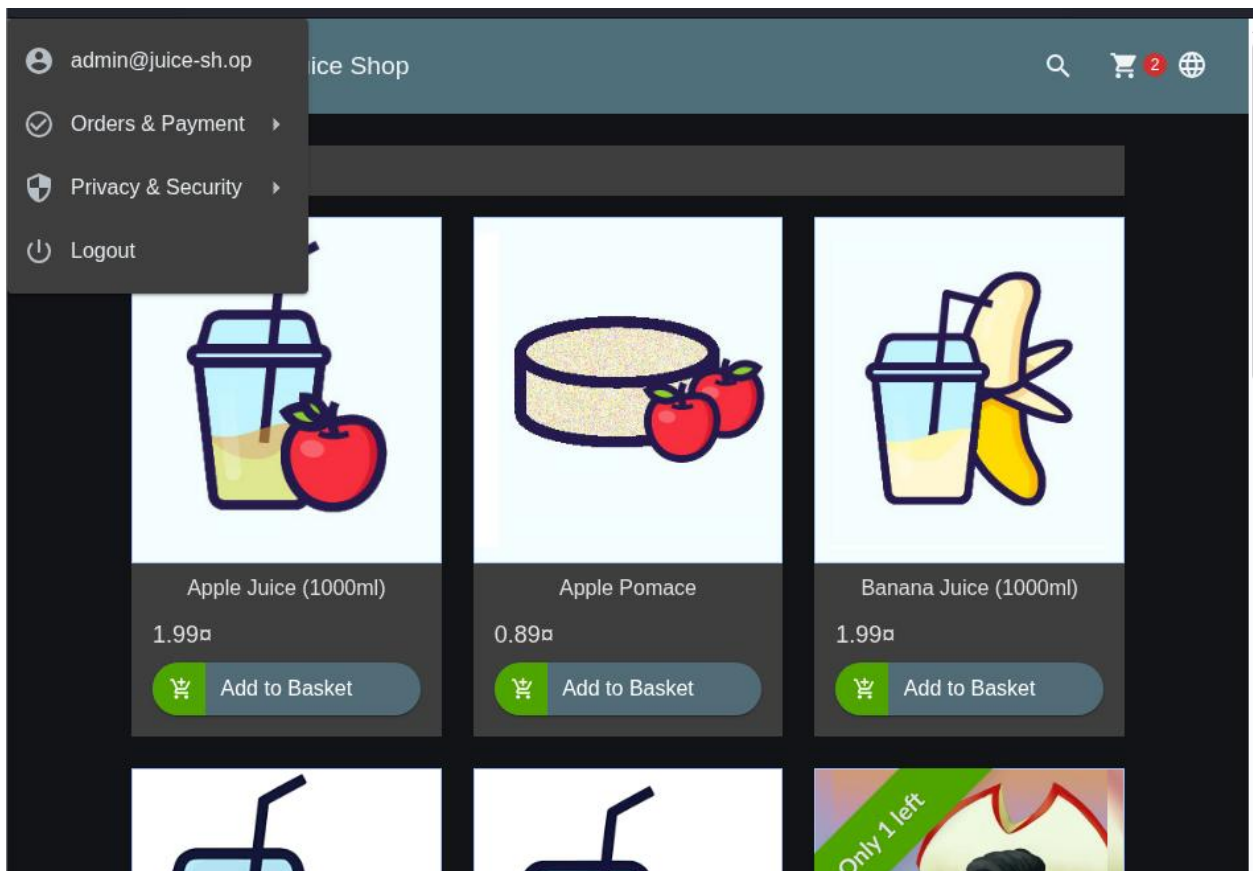


Figure 19 - Administrative user accessing a privileged application function

Pretty	Raw	Hex
--------	-----	-----

```

1 GET /api/Products/1?d=Web%20Sep%2002%202022 HTTP/1.1
2 Host: localhost:3000
3 sec-ch-ua-platform: "Linux"
4 Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoynSwidXNlcm5hbWUiOiIiLCJlbWVpbCI6InRodWxoYXJpYmFuZGFyYUbnbWVpbC5jb20iLCJwYXNkd29yZCI6IjIlZjllNzk0MzIzYjQ1Mzg4NWY1MTgzXjFiNjIOZDBiIiwicm9sZSI6ImNlciR3RvbnVvIiwidGVzsdXhlVG9rZW4iOiIiLCJ5XN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInRyZ2pjbWVnZSI6Ii9hc3NldmVmcHVibGljL2ltYWdlc3Y9IcGxvYWRzL2RlZmFibHouc3ZnIiwidG90cFNLy3JlZCI6IiIsImIzZWQWNaXzIjIjp0cnVlLCJjcmVhdGVkQXQ0IiOiIyMDI2LTAsLTAxIDA2IOjEOLjE0SARMDA6MDAiLCJlciRhdGVkQXQ0IiOiIyMDI2LTAsLTAxIDA4OjI3OjEwLjEzNSARMDA6MDAiLCJkZWxlZGVkQXQ0IiOm51bGx9L2JiaWQ0Ij0YsImIldCI6MTc4ODMyMzQ4Nn0i.XfiBSro2rgzGGW2tkHmhB-ysWzno_y46LZGHMBXmbAc027oTvRI30j0WuPTc9kRnYPSS0rJBdvzhWcjUq_ijHByeNuotHzcuscD-8qamyg48mEtZVIDaXvyZ4yk9PLQJRTRWkzJQfRcfqbtPj3uVlVikiOum-bYW2wjUNC92ig
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/plain, */*
7 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 sec-ch-ua-mobile: ?0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://localhost:3000/
14 Accept-Encoding: gzip, deflate, br
15 Cookie: language=en; welcombanner_status=dismiss; cookieconsent_status=dismiss; continueCode=PygMZE4p6L8NXYxJwoQ2bqjnG7LSRRhbRt790Ka3071brWD5yzrBemkPL9VX; token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoynSwidXNlcm5hbWUiOiIiLCJlbWVpbCI6InRodWxoYXJpYmFuZGFyYUbnbWVpbC5jb20iLCJwYXNkd29yZCI6IjIlZjllNzk0MzIzYjQ1Mzg4NWY1MTgzXjFiNjIOZDBiIiwicm9sZSI6ImNlciR3RvbnVvIiwidGVzsdXhlVG9rZW4iOiIiLCJ5XN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInRyZ2pjbWVnZSI6Ii9hc3NldmVmcHVibGljL2ltYWdlc3Y9IcGxvYWRzL2RlZmFibHouc3ZnIiwidG90cFNLy3JlZCI6IiIsImIzZWQWNaXzIjIjp0cnVlLCJjcmVhdGVkQXQ0IiOiIyMDI2LTAsLTAxIDA2IOjEOLjE0SARMDA6MDAiLCJlciRhdGVkQXQ0IiOiIyMDI2LTAsLTAxIDA4OjI3OjEwLjEzNSARMDA6MDAiLCJkZWxlZGVkQXQ0IiOm51bGx9L2JiaWQ0Ij0YsImIldCI6MTc4ODMyMzQ4Nn0i.XfiBSro2rgzGGW2tkHmhB-ysWzno_y46LZGHMBXmbAc027oTvRI30j0WuPTc9kRnYPSS0rJBdvzhWcjUq_ijHByeNuotHzcuscD-8qamyg48mEtZVIDaXvyZ4yk9PLQJRTRWkzJQfRcfqbtPj3uVlVikiOum-bYW2wjUNC92ig
16 Connection: keep-alive
17
18

```

Figure 22 - Replay of the privileged API request using a standard-user session

[illegible]

Figure 23 - Standard user successfully invoking a function restricted to administrative users

Finding: Broken Function-Level Authorization (BFLA)

The test demonstrated that a standard user was able to invoke a function that was expected to be restricted to administrative users. The administrative request returned HTTP 200, and replaying the corresponding request using a standard-user session also returned HTTP 200. This indicates insufficient server-side function-level authorization.

7.4 Injection

Assessment Objective

Identify application parameters that accept user-controlled input and assess whether improper validation or sanitization could allow injection.

Potential inputs include:

- Search parameters
- Query parameters
- JSON fields
- Form inputs
- URL parameters

Initial testing should be controlled and non-destructive.

Testing Records

Table 8 - Injection Test Records

Test ID	Test Case	Injection Point	Payload / Input	Expected Result	Actual Result	Status
INJ-01	Baseline request	Login / input parameter	Normal valid input	Application processes request normally	Normal response received	Passed
INJ-02	Special-character test	Login email field	'	Application should safely handle special characters without database errors	Record your response	Passed
INJ-03	SQL injection test	Login email field	' OR '1'=1	Application should reject the input and authentication should fail	Record your response	Vulnerable

Pretty	Raw	Hex
--------	-----	-----

```

1 GET /rest/products/search?q=apple HTTP/1.1
2 Host: 127.0.0.1:3000
3 sec-ch-ua-platform: "Linux"
4 Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpp7ImIkIjoxLjCjL2VybWFTZSI6IiIsImVtYWlsIjoiYWRtaW5AanVpY2U
tC2gub3AiLCJwYXNzd29yZCI6IjAxOTIwMjNhN2JiZDczMjUwNTE2ZjA2OWRmMThtNTAwIiwicm9sZSI6ImFkbWluIiwiaGVhZGVz
d29yZCI6IiLCJscXN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInByb2ZpbGVJbWFnZSI6IiIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdGJZSI
6dHJlZSwiY3JlYXRlZEF0IjoiMjAyNi4wOS0wMSAwNjoxMT0yOC45NDMgKzAwOjAwIiwidXBkYXRlZEF0IjoiMjAyNi4wOS0wMSAwNT
wMT0lMS41MjUgKzAwOjAwIiwiaGVhZGVzZXRLZEF0IjppudWxsfsWmIkIjoxLjCjYpYXQ1OjE3ODgzMjg4NjV9.wrPhTMFAAHUg4DwrOtPToz
hGDG0HZaw-kV0UwTZGARy9_NjWeM8wwGr64EpZ58TnHoLK3zd8Spn7NoDtBzb87Dpj3g_hz82fL4WcYDQFsw7MLgsy391NtbTqhJ8fyVE
NFW0LXdo5LJe-StXW4hly4zw-3DhccB2K3V79C9glAU4
5 Accept-Language: en-US,en;q=0.9
6 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
7 sec-ch-ua-mobile: ?0
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0
Safari/537.36
9 Accept: application/json, text/plain, */*
10 X-User-Email: admin@juice-sh.op
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Dest: empty
14 Referer: http://127.0.0.1:3000/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpp7ImIkIjoxLjCjL2VybWFTZSI6IiIsImVtYWlsIjoiYWRtaW5AanVpY2U
tC2gub3AiLCJwYXNzd29yZCI6IjAxOTIwMjNhN2JiZDczMjUwNTE2ZjA2OWRmMThtNTAwIiwicm9sZSI6ImFkbWluIiwiaGVhZGVz
d29yZCI6IiLCJscXN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInByb2ZpbGVJbWFnZSI6IiIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdGJZSI
6dHJlZSwiY3JlYXRlZEF0IjoiMjAyNi4wOS0wMSAwNjoxMT0yOC45NDMgKzAwOjAwIiwidXBkYXRlZEF0IjoiMjAyNi4wOS0wMSAwNT
wMT0lMS41MjUgKzAwOjAwIiwiaGVhZGVzZXRLZEF0IjppudWxsfsWmIkIjoxLjCjYpYXQ1OjE3ODgzMjg4NjV9.wrPhTMFAAHUg4DwrOtPToz
hGDG0HZaw-kV0UwTZGARy9_NjWeM8wwGr64EpZ58TnHoLK3zd8Spn7NoDtBzb87Dpj3g_hz82fL4WcYDQFsw7MLgsy391NtbTqhJ8fyVE
NFW0LXdo5LJe-StXW4hly4zw-3DhccB2K3V79C9glAU4
17 Connection: keep-alive
18
19

```

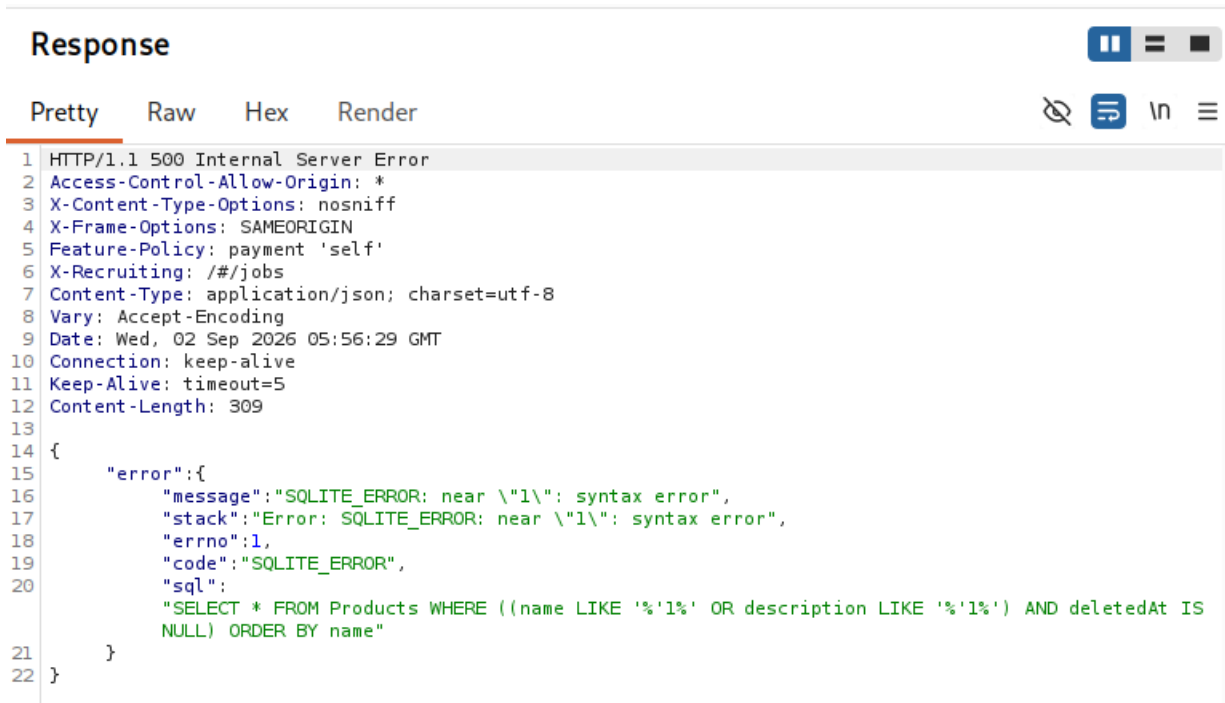
32

```

Request
  Ver  Method  Path  Status  Size  Time  Type  Size  Time  Type
  ---  ---  ---  ---  ---  ---  ---  ---  ---  ---
  1  GET  /rest/products/search?q='1 HTTP/1.1
  2  Host: 127.0.0.1:3000
  3  sec-ch-ua-platform: "Linux"
  4  Authorization: Bearer
    eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjP7ImIkIjoYNSwidXNlcm5hbWUiOiIiLCJlbWFPbCI6InRodWxoYXJpYmF
    uZGFyYUbnbWFPbC5jb20iLCJwYXNzd29yZCI6IjI1ZjllNzk0MzIzYjQ1Mzg4NWY1MTgxZjFiNjI0ZDBiIiwicm9sZSI6ImNlcnRvbnVW
    yIiwidGVzdXhlVG9rZW4iOiIiLCJ5YXN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2L
    tYWdlcy9lcGxvYWRzL2RlZmFlbHouc3ZnIiwidG90cFNLy3JldCI6IiIsImIzZW50aXZlIjP0cnVlLCJjcmVhdGvKXQXQiOiIyMDI2LTA
    SLTAxIDA20jI10jE0Ljcl0SARMDA6MDAiLCJlcGRhdGVkQXQiOiIyMDI2LTA5LTAxIDA40jI30jEwLjEzNSARMDA6MDAiLCJkZWxlZGV
    kQXQiOm5lbGx9LCAjiaWQiojYsImldhdCI6MTc4ODMyNzkxNH0.t3hijlZ2-g4ouJWwYlt5M8XggLlCiLkClG9CY2ztSSBFVsvk9gRFV5X
    pxX-mQkPurGyxlVl363QMndNckwflFsqafFTiohyiYIhHVuDAqxLSQe3IqWxiLzcGZp1jJm3Qwwe9w1rbNP5BpUm_4H2ze0HvhaN8zF
    HD_lsXiJz4pM
  5  Accept-Language: en-US,en;q=0.9
  6  sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
  7  sec-ch-ua-mobile: ?0
  8  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0
    Safari/537.36
  9  Accept: application/json, text/plain, */*
  10 X-User-Email: thulharibandara@gmail.com
  11 Sec-Fetch-Site: same-origin
  12 Sec-Fetch-Mode: cors
  13 Sec-Fetch-Dest: empty
  14 Referer: http://127.0.0.1:3000/
  15 Accept-Encoding: gzip, deflate, br
  16 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; token=
    eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjP7ImIkIjoYNSwidXNlcm5hbWUiOiIiLCJlbWFPbCI6InRodWxoYXJpYmF
    uZGFyYUbnbWFPbC5jb20iLCJwYXNzd29yZCI6IjI1ZjllNzk0MzIzYjQ1Mzg4NWY1MTgxZjFiNjI0ZDBiIiwicm9sZSI6ImNlcnRvbnVW
    yIiwidGVzdXhlVG9rZW4iOiIiLCJ5YXN0TG9naW5JcCI6IjEwLjAuMi4xNSIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2L
    tYWdlcy9lcGxvYWRzL2RlZmFlbHouc3ZnIiwidG90cFNLy3JldCI6IiIsImIzZW50aXZlIjP0cnVlLCJjcmVhdGvKXQXQiOiIyMDI2LTA
    SLTAxIDA20jI10jE0Ljcl0SARMDA6MDAiLCJlcGRhdGVkQXQiOiIyMDI2LTA5LTAxIDA40jI30jEwLjEzNSARMDA6MDAiLCJkZWxlZGV
    kQXQiOm5lbGx9LCAjiaWQiojYsImldhdCI6MTc4ODMyNzkxNH0.t3hijlZ2-g4ouJWwYlt5M8XggLlCiLkClG9CY2ztSSBFVsvk9gRFV5X
    pxX-mQkPurGyxlVl363QMndNckwflFsqafFTiohyiYIhHVuDAqxLSQe3IqWxiLzcGZp1jJm3Qwwe9w1rbNP5BpUm_4H2ze0HvhaN8zF
    HD_lsXiJz4pM
  17 Connection: keep-alive
  18

```

Figure 25 - Injection test payload submitted through the API parameter



```
1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Vary: Accept-Encoding
9 Date: Wed, 02 Sep 2026 05:56:29 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12 Content-Length: 309
13
14 {
15   "error":{
16     "message":"SQLITE_ERROR: near \"1\": syntax error",
17     "stack":"Error: SQLITE_ERROR: near \"1\": syntax error",
18     "errno":1,
19     "code":"SQLITE_ERROR",
20     "sql":
21       "SELECT * FROM Products WHERE ((name LIKE '%1%' OR description LIKE '%1%')) AND deletedAt IS
22       NULL) ORDER BY name"
```

Figure 26 - API response following injection test

7.5 Security Misconfiguration

Assessment Objective

Identify configuration weaknesses such as:

- Excessive error information
- Missing security controls
- Unnecessary exposed functionality
- Weak HTTP security configuration
- Debug information
- Insecure defaults

Evidence

The HTTP response headers were reviewed to identify missing or improperly configured security-related headers.

The application also returned detailed error information that exposed internal application details.


```

1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 30
9 ETag: W/"1e-JkPcI+pGj7BBTx0uZTVVIm91zaY"
10 Vary: Accept-Encoding
11 Date: Wed, 02 Sep 2026 06:28:22 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
16   "status": "success",
17   "data": [
18   ]
19 }

```

Figure 29 - Security-related HTTP header configuration identified during testing

8. Risk Assessment and CVSS Scoring

Only confirmed vulnerabilities will receive final severity ratings.

8.1 Risk Classification

Table 9 - Risk Classification

Severity	General Interpretation
Critical	Severe compromise with significant impact
High	Significant unauthorized access or data/security impact
Medium	Meaningful security weakness with limited or conditional impact
Low	Limited security impact
Informational	Security observation without direct exploitable impact

8.2 Risk Registration

Table 10 - Risk Registration

ID	Finding	Severity	CVSS v3.1	Justification	Remediation Priority
WEB-001	BOLA / IDOR	High	7.1	A low-privileged authenticated user may access another user's object by manipulating an object identifier,	P1 – Immediate

				potentially exposing or modifying unauthorized data.	
WEB-002	Broken Authentication	High	7.5	Weaknesses in authentication controls may allow unauthorized access to protected functionality or accounts.	P1 – Immediate
WEB-003	Broken Function-Level Authorization	High	7.6	A lower-privileged user may invoke functionality intended for a higher-privileged role, potentially resulting in unauthorized administrative actions.	P1 – Immediate
WEB-004	Excessive Data Exposure	Medium	5.3	API responses may disclose unnecessary security-sensitive or internal information beyond the application's functional requirements.	P2 – High
WEB-005	Injection	Critical	9.8	Successful injection can allow manipulation of backend queries or commands and potentially lead to unauthorized data access or modification.	P1 – Immediate
WEB-006	Security Misconfiguration	Low	3.7	Insecure configuration or unnecessary information disclosure can provide attackers with additional information or weaken defensive controls.	P3 – Planned

CVSS scores will be calculated after each finding has been technically validated.

Table 11 - CVSS scores

Priority	Meaning
P1 – Immediate	Remediate as soon as possible; significant compromise or unauthorized access is demonstrated.
P2 – High	Remediate in the near term; meaningful security exposure exists.
P3 – Planned	Address during normal security hardening.
P4 – Informational	Best-practice improvement with limited direct security impact.

9. Remediation Recommendations

The following remediation recommendations address the security weaknesses identified during the assessment.

BOLA / IDOR

Recommended controls include:

- Implement server-side object-level authorization.
- Verify that the authenticated user owns or has permission to access the requested object.
- Do not rely on unpredictable object identifiers as a security control.
- Return appropriate authorization responses.
- Implement automated authorization tests.

Broken Authentication

Recommended controls include:

- Implement robust authentication controls.
- Secure session management.
- Validate authentication tokens server-side.
- Enforce appropriate password policies.
- Prevent authentication bypass conditions.
- Invalidate sessions appropriately.

Broken Function-Level Authorization

Recommended controls include:

- Enforce authorization server-side.
- Apply role-based access control.
- Verify privileges for every sensitive operation.
- Do not rely on hidden UI elements for access control.

Injection

Recommended controls include:

- Validate user input.
- Apply context-appropriate output encoding.
- Use parameterized queries where applicable.
- Avoid unsafe dynamic query construction.

Security Misconfiguration

Recommended controls include:

- Disable unnecessary debugging information.
- Configure appropriate security headers.
- Remove unnecessary exposed functionality.
- Implement secure application defaults.
- Review server and application configurations regularly.

10. Conclusion

This Web Application and API Security Assessment was conducted against OWASP Juice Shop within an authorized local laboratory environment.

The assessment established a structured security-testing process beginning with target verification and reconnaissance. The Juice Shop application was successfully deployed using Podman and accessed through its local HTTP interface. Nmap was used to identify the exposed service, while Burp Suite was configured to intercept and analyze application traffic.

API enumeration was performed by interacting with the application and reviewing HTTP requests through Burp Suite HTTP History and Site Map. This process provided visibility into the application's endpoints, methods, parameters, authentication information, and API responses.

The assessment then prioritized endpoints according to potential security risks, with particular emphasis on object-level authorization, authentication, function-level authorization, data exposure, injection, and security configuration.

The vulnerability assessment is designed around manual validation and reproducible evidence. This ensures that final findings represent genuine security weaknesses rather than assumptions based solely on endpoint discovery or automated observations.

The confirmed vulnerabilities identified during the assessment were evaluated according to their potential impact and exploitability. Appropriate severity classifications and CVSS scores were assigned, and remediation recommendations were provided for each confirmed finding.

The project therefore demonstrates a practical penetration-testing workflow covering reconnaissance, API enumeration, vulnerability validation, risk assessment, and professional security reporting.